

SCUBEX: HERRAMIENTA Y RED SOCIAL PARA BUCEADORES

IVÁN FERNÁNDEZ LIMÁRQUEZ

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo



Universidad de Sevilla

abril 2026

Publicado en abril 2026 por

Iván Fernández Limárquez

Copyright © MMXXVI

<http://www.lsi.us.es/~trinidad>

ivaferlim@alums.es

Pon aquí cuestiones acerca del copyright

Yo, D. Iván Fernández Limárquez con NIF número 77015962W,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

Scubex: Herramienta y Red social para Buceadores

Lo cual firmo,

Fdo. D. Iván Fernández Limárquez
en la Universidad de Sevilla
22/04/2026

Para mi familia, amigos y para el mar que ha inspirado este proyecto.



AGRADECIMIENTOS

Agradecer a mi familia, amigos y conocidos que han probado mi aplicación. Con los que pese a haber sido tan pesado, han tenido la paciencia de seguir mis interesantísimas discusiones sobre temas como: Que color es mas fácil de ver, donde es objetivamente mas intuitivo que vaya un botón o, por vigesimotercera vez, escucharme sobre lo chula que esta la animación de inicio.

Dedicado a mi madre, quien sabe cuanto amo el mar y me ha impulsado y apoyado en todo momento.

RESUMEN

Scubex es una red social y herramienta pensada para buceadores que necesiten conocer diversos aspectos relacionados con las zonas de inmersión. La aplicación permite ver las especies en la zona y datos sobre ellas, condiciones climatológicas y compartir experiencias sobre la misma. Mediante publicaciones los usuarios podrán conectar unos con otros y conocer de la mano de demás buceadores si esa zona es interesante o no.

ÍNDICE GENERAL

I	Introducción	1
1.	Contexto	3
1.1.	Redes sociales y exploración marina	4
1.2.	Aplicaciones y tecnologías similares	4
2.	Objetivos	7
2.1.	Motivación	8
2.2.	Listado de objetivos	8
II	Organización del proyecto	11
3.	Metodología	13
3.1.	Estructura organizacional del proyecto	14
3.2.	Metodología de desarrollo	14
3.2.1.	Justificación de Feature-Driven Development (FDD)	14
3.2.2.	Adaptación del ciclo FDD para Scubex	15
3.2.3.	Herramientas de desarrollo	15
3.3.	Seguimiento y control	16
4.	Planificación	17

4.1. Resumen temporal del proyecto	18
4.2. Planificación inicial	18
4.3. Informe de tiempos del proyecto	18
5. Costes	21
5.1. Resumen de costes del proyecto	22
5.2. Costes de personal	22
5.2.1. Desarrollador Full-Stack (Arquitectura e Implementación)	22
5.2.2. Becario Junior (Redacción de Documentación)	23
5.2.3. Total costes de personal	23
5.3. Costes materiales	24
5.3.1. Amortización de equipamiento informático	24
5.3.2. Despliegue en la nube	24
5.3.3. Total costes materiales	25
5.4. Costes indirectos	25
5.5. Coste total del proyecto	25
III Desarrollo del proyecto	27
6. Arranque	29
6.1. Lista de características	30
6.2. Diseño arquitectónico	31
6.2.1. Stack Tecnológico	31
6.2.2. Estrategia de entornos	32
6.2.3. Justificación de MapLibre GL JS + MapTiler	33
6.2.4. Integración de APIs Externas	33

6.2.5. Patrón Arquitectónico: Gateway API	34
6.3. Seguimiento y control	35
7. Iteración 1: Escáner de Biodiversidad	37
7.1. Características desarrolladas esta iteración	38
7.2. Diseño	38
7.2.1. Incompatibilidad 1: Búsqueda radial vs. geométrica	38
7.2.2. Incompatibilidad 2: Calidad de datos visuales	39
7.2.3. Uso del campo <code>phy1um</code> como fallback	40
7.3. Implementación	40
7.4. Pruebas	41
7.4.1. Caché de escaneos (resolución de deuda técnica)	41
7.4.2. Documentación de API (Swagger/OpenAPI)	42
7.4.3. Integración frontend (TD2)	42
7.4.4. Configuración del proyecto (TD3)	43
7.5. Despliegue	44
7.6. Seguimiento y control	44
IV Cierre del proyecto	51
8. Manual de usuario	53
8.1. Sección libre	54
9. Conclusiones	55
9.1. Informe post-mortem	56
9.1.1. Lo que ha ido bien	56

9.1.2. Lo que ha ido mal	56
9.1.3. Discusión	56
9.2. Trabajos futuros	56
V Appendices	57
A. Software Product Lines	59
A.1. Software Product Lines	60
A.2. Feature Models	60
A.3. Automated Analysis of Feature Models	62
A.3.1. Scope	62
A.4. Dynamic Software Product Lines (DSPL)	65
A.5. Hypothesis and Objectives	65
Referencias bibliográficas	68

ÍNDICE DE FIGURAS

A.1. An example of a Home Integration System	61
A.2. A different view on AAFM distinguishing between information extrac- tion and explanatory operations	63

ÍNDICE DE CUADROS

4.1. Tabla resumen de tiempos y planificación	18
4.2. Planificación temporal de iteraciones	18
4.3. Planificación temporal de iteraciones	19
5.1. Tabla resumen de costes del proyecto	22
5.2. Coste total del proyecto Scubex	25
6.1. Estrategia de entornos del proyecto	32
6.2. Resumen de APIs externas	34
7.1. Análisis de valor aportado: Escáner	46
7.2. Memorando técnico 001: Geometría	47
7.3. Memorando técnico 002: Filtrado	48
7.4. Memorando técnico 003: Caché de Escaneos	49
A.1. Most frequently used explanatory operations and their corresponding information extraction operations	67

LISTA DE TAREAS PENDIENTES

■ To Abductive Section in 2.1	60
Figura: A feature model example	61
■ To Abductive Intro	62

PARTE I

INTRODUCCIÓN

CONTEXTO

The sea, once it casts its spell, holds one in its net of wonder forever.

*Jacques Yves Cousteau,
Oceanographer*

Vamos a dar una visión general del contexto en el que se desarrolla el proyecto, incluyendo el estado actual de la industria y el porqué del mismo.

1.1 REDES SOCIALES Y EXPLORACIÓN MARINA

Desde pequeño he vivido rodeado de mar, pero es solo ahora cuando he descubierto lo mucho que me gusta el buceo. Sin embargo, este viene con una serie de retos y problemas: ¿Dónde me sumerjo?, ¿Hará buen clima debajo del agua?, ¿Podré llegar a la zona con mi equipo o hará mucho viento? y, si voy, ¿Qué especies podré ver?

Con este proyecto busco dar una solución a estos problemas desde dos perspectivas. Estadística y personal, el usuario podrá basarse en cientos de datos ya recopilados de una zona, tomado de una base de datos que lleva creada y actualizándose desde el 1970. O simplemente podrá basarse en la experiencia de otros buceadores en esa misma zona.

1.2 APLICACIONES Y TECNOLOGÍAS SIMILARES

No tendría sentido desarrollar un proyecto que ya existe, por ello he buscado aplicaciones que hagan algo similar a la propuesta de Scubex. Que básicamente se basa en dar respuesta a las preguntas preguntas que he mencionado, para ello hay que dar información de: Que condiciones climatológicas hay en una zona, que especies marinas podrás ver y como está el mar ese día concreto en esa zona específica. Esto debe hacerse usando tanto datos recopilados a lo largo de los años de una zona, como dejando a los usuarios compartir sus experiencias personales. Buscando aplicaciones que den respuesta a estas preguntas he encontrado tres:

La primera es **Dive Mate**: Una aplicación que te muestra especies que has visto tú en una zona y que te da estimaciones climatológicas precisas. Sin embargo, la interfaz es tosca. Y no es posible compartir experiencias personales, un usuario no puede interactuar con otros ni registrar sus propias observaciones.

La segunda es **iNaturalist**: Una aplicación que te permite ver, en un mapa, avistamientos de especies en esa zona. Sus fotos e información de avistamientos son increíblemente ricas y útiles. Sin embargo, no está centralizada en buceadores y no incluye información climatológica para el mar de la zona.

La última es **DiveApp**: Una aplicación orientada a los avistamientos y preparación de viajes que incluye componentes de red social. Tiene el problema contrario a Dive Mate, incluye experiencia personal a modo de publicaciones y clima sobre la zona. Pero sin embargo no incluye datos ya recopilados sobre la zona en la que pretendes

sumergirte.

Como conclusión, hay aplicaciones que ofrecen funcionalidades parecidas a las que esta aplicación proporciona, sin embargo, ninguna las centraliza. Ni están implementadas de una forma intuitiva, ni con tecnologías modernas. En este hueco de mercado encaja Scubex. Por ello me he decidido a hacer este proyecto

OBJETIVOS

The sea, once it casts its spell, holds one in its net of wonder forever.

*Jacques Yves Cousteau,
Oceanographer*

Visto el contexto, formalizaremos los objetivos de la app de forma concreta. ¿Hacia quien esta dirigida la aplicación? ¿Qué funcionalidades tendrá? ¿Qué tecnologías se van a usar? ¿Qué se va a conseguir con el proyecto?

2.1 MOTIVACIÓN

Como ya hemos comentado, la idea de Scubex es centralizar funcionalidades para el nicho del buceo. Para lograrlo se han planteado esta serie de objetivos que, una vez cumplidos, darán como resultado la aplicación deseada.

2.2 LISTADO DE OBJETIVOS

Objetivo 1. Implementar un sistema de autenticación de usuarios

Desarrollar un sistema de autenticación seguro, permitiendo a los usuarios registrarse e iniciar sesión de forma sencilla. El sistema debe guardar los perfiles de usuario y datos tales como: nombre, foto, incursiones o publicaciones.

Objetivo 2. Crear un mapa interactivo de exploración marina

Integrar un mapa interactivo con controles de zoom y navegación que permita a los usuarios explorar zonas de buceo de forma intuitiva y fluida.

Objetivo 3. Integrar datos científicos de biodiversidad marina

Desarrollar un escáner de especies que consulte APIs científicas para identificar fauna marina en una zona determinada.

Objetivo 4. Proporcionar información climatológica y oceanográfica en tiempo real

Ofrecer datos actualizados sobre condiciones climatológicas de la zona: viento, temperatura del agua, corrientes marinas, oleaje y demás datos atmosféricos relevantes para la planificación de inmersiones en una zona de inmersión.

Objetivo 5. Implementar un sistema de registro de publicaciones

Permitir a los usuarios publicar sus inmersiones en el mapa interactivo con título, descripción opcional, fotografía y coordenadas geográficas. El autor puede editar y eliminar sus propias publicaciones, y cada publicación es compartible mediante enlace directo.

Objetivo 6. Crear un sistema de interacción social sobre publicaciones

Desarrollar un sistema de interacciones sobre publicaciones: likes, guardado para acceso posterior y comentarios con menciones a otros usuarios. Las menciones generan notificaciones al usuario mencionado.

Objetivo 7. Implementar una red social de seguimiento entre usuarios

Desarrollar perfiles públicos navegables con número de seguidores y seguidos,

posibilidad de seguir o dejar de seguir a cualquier usuario, y acceso al historial de publicaciones de cada perfil. Los perfiles son compartibles mediante enlace directo.

Objetivo 8. Implementar un sistema de notificaciones

Proporcionar notificaciones al usuario sobre eventos relevantes: nuevos seguidores, likes, comentarios en sus publicaciones y menciones. Las notificaciones son gestionables, pudiendo borrarlas o marcarlas como leídas.

PARTE II

ORGANIZACIÓN DEL PROYECTO

METODOLOGÍA

Don't reinvent the wheel, just realign it.

*Anthony J. D'Angelo,
Escritor motivacional*

***E**ste capítulo describe el marco metodológico adoptado para el desarrollo de Scubex, basado en Feature-Driven Development (FDD) con adaptaciones específicas para un proyecto académico individual.*

3.1 ESTRUCTURA ORGANIZACIONAL DEL PROYECTO

Scubex se ha desarrollado como Trabajo Fin de Grado de forma individual, asumiendo las siguientes responsabilidades:

- **Arquitecto de Software:** Diseño de la arquitectura cliente-servidor y selección del stack tecnológico.
- **Desarrollador Full-Stack:** Implementación del backend (Spring Boot) y frontend (React + Vite).
- **Documentador Técnico:** Redacción de la memoria académica y especificación de APIs mediante OpenAPI (Swagger).

El tutor del proyecto ha proporcionado la supervisión técnica y académica, validando decisiones arquitectónicas y avances iterativos.

3.2 METODOLOGÍA DE DESARROLLO

3.2.1 Justificación de Feature-Driven Development (FDD)

Se ha adoptado **Feature-Driven Development** como metodología central debido a sus características idóneas para proyectos de alcance medio con requisitos funcionales bien definidos:

- **Orientación a características:** Cada funcionalidad (escáner de especies, integración climática, red social) se desarrolla como una única unidad. Lo que significa que se realizará el código y hasta que no esté documentada y acabada, no se pasará a la siguiente.
- **Trazabilidad:** Al forzar la compleción completa de una característica hay una relación estrecha y directa entre requisitos, código y documentación. Esto hace que sea fácil trazar un camino claro del avance del proyecto.
- **Escalabilidad:** Gracias a este enfoque es relativamente añadir nuevas características en el futuro. Ya que simplemente hay que añadir una característica a la lista y tratarla como esa unidad que previamente hemos mencionado.

En resumen, esta metodología permite estructurar el desarrollo con una lista de características claras y priorizadas. Lo que ayuda a mantener una organización coherente y simple en un equipo pequeño o individual.

3.2.2 Adaptación del ciclo FDD para Scubex

El modelo clásico de FDD consta de 5 fases. Para este proyecto académico se ha adaptado la secuencia, unificando la fase de diseño y añadiendo una fase de documentación:

1. **Desarrollar la lista de características:** Identificación y priorización de funcionalidades nucleares (Gestión de usuarios OAuth2, Escáner de especies, Exploración climática, Red social).
2. **Construir un modelo global:** Diseño de la arquitectura del sistema a partir de las características identificadas (capítulo de Arranque).
3. **Planificar por característica:** Estimación tecnológica, limitaciones y consecuente diseño del flujo de datos.
4. **Construir por característica:** Implementación incremental con validación mediante pruebas constantes.
5. **Documentar la característica:** Redacción académica de cada característica en LaTeX, incluyendo justificación de decisiones técnicas y lecciones aprendidas.

3.2.3 Herramientas de desarrollo

- **Control de versiones:** Git + GitHub (commits atómicos por característica).
- **Gestión de dependencias:** Maven (backend), npm (frontend).
- **Testing:** JUnit 5 + Mockito (backend), Vitest (frontend).
- **Documentación de API:** Swagger/OpenAPI 3.0 (generación automática de contratos REST).
- **Compilación:** Vite (desarrollo frontend con HMR), Spring Boot Maven Plugin (empaquetado JAR).

- **Despliegue:** Vercel (frontend, despliegue automático por rama con previews) + Railway (backend Spring Boot y PostgreSQL, entornos de preproducción y producción independientes).

3.3 SEGUIMIENTO Y CONTROL

El progreso del proyecto se ha monitorizado mediante:

- **Archivo TODO:** Lista priorizada de tareas pendientes con urgencias marcadas (!).
- **Commits semánticos:** Mensajes (casi siempre) estructurados (feat, fix, docs, refactor) para trazabilidad histórica.
- **Reuniones de seguimiento:** Sesiones cada tres semanas con el tutor para validar decisiones arquitectónicas y resolver bloqueos técnicos.
- **Documentación incremental:** Redacción de capítulos inmediatamente tras completar cada característica dentro de la aplicación.

Métricas de avance:

- Características completadas.
- Cobertura de tests unitarios (objetivo: >70% en lógica de negocio).

PLANIFICACIÓN

Divide y vencerás · El arte de la guerra

*Sun Tzu (544–496 a.C.),
Maestro de guerra y escritor*

*E*n este capítulo se exponen las divisiones en sprint planificadas para la realización del proyecto.

4.1 RESUMEN TEMPORAL DEL PROYECTO

Resumen del proyecto	
Fecha de inicio	13/10/2025
Fecha de fin	XX/XX/2026
Periodicidad de las revisiones	3 semanas
Carga de trabajo semanal	12 horas
Horas totales previstas	225 horas
Horas finales	XXX horas

Cuadro 4.1: Tabla resumen de tiempos y planificación

4.2 PLANIFICACIÓN INICIAL

Aquí un desglose de las iteraciones, comienzo y fin de cada una:

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 08/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.2: Planificación temporal de iteraciones

Cabe destacar que la primera iteración es más larga que las siguientes. Esto se debe a que el proyecto se inició en octubre, mientras que la documentación comenzó en febrero. Dado que hubo un parón entre ambas fechas, se decidió que la primera iteración abarcara todo ese tiempo, para no perder el trabajo realizado durante ese periodo. Las siguientes iteraciones se planificaron con una duración de 3 semanas cada una, para mantener un ritmo constante de trabajo y permitir revisiones periódicas del progreso del proyecto.

4.3 INFORME DE TIEMPOS DEL PROYECTO

Lo mismo que el anterior pero con datos reales. Ver Tabla §4.3.

Resumen de iteraciones	
Iteración 1	13/10/25 a 13/03/26
Iteración 2	13/03/26 a 03/04/26
Iteración 3	04/04/26 a 24/04/26
Iteración 4	25/04/26 a 08/05/26
...	dd/mm/aa a dd/mm/aa

Cuadro 4.3: Planificación temporal de iteraciones

Tras la primera iteración, no hubo más descoordinación entre las fechas planificadas y las reales. Se cumplió con el calendario establecido.

COSTES

Here Comes the Money

*Jim Johnston y Naughty by Nature,
Escritores y Raperos*

***E**n este capítulo se realiza el desglose de los costes asociados al proyecto, basándose en los salarios reales de puestos tecnológicos.*

5.1 RESUMEN DE COSTES DEL PROYECTO

Resumen del proyecto	
Costes de personal	14.950 €
Desarrollador Full-Stack	9.750 €
Becario Documentación	5.200 €
Costes materiales	68,40 €
Costes indirectos	1.500 €
TOTAL	16.518,40 €

Cuadro 5.1: Tabla resumen de costes del proyecto

5.2 COSTES DE PERSONAL

Para el cálculo de los costes de personal se han consultado los datos de la **Guía Salarial 2026** de Manfred¹, que recopila información salarial de más de 120.000 profesionales de tecnología en España.

Dado que el desarrollo de Scubex ha requerido asumir múltiples roles y tareas, se han considerado dos perfiles:

5.2.1 Desarrollador Full-Stack (Arquitectura e Implementación)

Se ha considerado como referencia el salario de **Desarrollador Full-Stack** con experiencia de 5-10 años, cuyo rango salarial se sitúa entre 41.000-50.000€ anuales según dicha guía. Se ha tomado un valor medio de **45.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

La carga de trabajo estimada para el TFG es de **300 horas**, frente a una jornada laboral anual estándar de **1.800 horas**. Por tanto, el coste proporcional se calcula como:

$$\text{Salario proporcional} = \frac{45.000\text{€}}{6} = 7.500\text{€} \quad (5.1)$$

¹<https://www.getmanfred.com/blog/guia-salarial-2026-salarios-en-tecnologia-espana-manfred>

Costes sociales

A este salario bruto se le añaden los **costes sociales** (Seguridad Social, contingencias comunes, desempleo, formación profesional, etc.) que suponen aproximadamente el **30 %** del salario bruto:

$$\text{Costes sociales} = 7,500\text{€} \times 0,30 = 2,250\text{€} \quad (5.2)$$

Total Desarrollador Full-Stack

$$\text{Total} = 7,500\text{€} + 2,250\text{€} = 9,750\text{€} \quad (5.3)$$

5.2.2 Becario Junior (Redacción de Documentación)

Para la redacción de la memoria académica y documentación técnica se ha considerado el perfil de un **becario junior** con experiencia menor a 2 años. Según la guía salarial, los desarrolladores junior se sitúan en el rango de 20.000-30.000€ anuales. Se ha tomado un valor de **24.000€ brutos anuales**.

Cálculo proporcional al tiempo dedicado

De la misma forma que se ha calculado anteriormente:

$$\text{Salario proporcional} = \frac{24,000\text{€}}{6} = 4,000\text{€} \quad (5.4)$$

Costes sociales

$$\text{Costes sociales} = 4,000\text{€} \times 0,30 = 1,200\text{€} \quad (5.5)$$

Total Becario

$$\text{Total} = 4,000\text{€} + 1,200\text{€} = 5,200\text{€} \quad (5.6)$$

5.2.3 Total costes de personal

$$\text{Total} = 9,750\text{€} + 5,200\text{€} = 14,950\text{€} \quad (5.7)$$

5.3 COSTES MATERIALES

5.3.1 Amortización de equipamiento informático

Según el criterio de amortización fiscal establecido por la Agencia Tributaria española, los equipos informáticos se amortizan a un **máximo del 25 % anual** (vida útil de 4 años).

Para el desarrollo del proyecto se ha utilizado un portátil con un coste de adquisición de **1.200€**:

- **Amortización anual:** $1,200€ \times 0,25 = 300€$
- **Amortización proporcional** (300h / 1.800h): $300€ \times \frac{300}{1,800} = 50€$

5.3.2 Despliegue en la nube

El proyecto se encuentra desplegado en producción y preproducción mediante dos servicios:

- **Railway** (backend Spring Boot + PostgreSQL): Plan Hobby a 5 USD/mes (~4,60 €/mes). Este plan incluye 5 USD de créditos de uso mensuales; superado ese umbral, se factura por consumo real de recursos. Actualmente debido al tráfico bajo-moderado, el crédito mensual de 5 USD resulta suficiente.
- **Vercel** (frontend React/Vite): Plan Hobby gratuito, sin coste.

Coste estimado de despliegue durante el desarrollo del TFG (4 meses):

$$\text{Railway} = 4,60 \text{ €/mes} \times 4 \text{ meses} = 18,40 \text{ €} \quad (5.8)$$

$$\text{Total despliegue} = 18,40 \text{ €} + 0 \text{ € (Vercel)} = 18,40 \text{ €} \quad (5.9)$$

En un escenario de producción real con mayor tráfico, los costes de Railway podrían aumentar proporcionalmente al uso de CPU, RAM y almacenamiento consumidos más allá del crédito incluido. En este caso podría crearse un plan personalizado, que incluya las necesidades concretas de la aplicación.

5.3.3 Total costes materiales

$$\text{Total} = 50 \text{ €} + 18,40 \text{ €} = 68,40 \text{ €} \quad (5.10)$$

5.4 COSTES INDIRECTOS

Los costes indirectos incluyen gastos generales asociados al desarrollo del proyecto que no se imputan directamente: electricidad, conexión a internet, espacio de trabajo, cuotas mensuales de asistentes de IA, etc.

Se ha tomado un valor medio del 10 % de la suma de costes de personal y materiales:

$$\text{Costes indirectos} = (14,950 \text{ €} + 68,40 \text{ €}) \times 0,10 = 1,501,84 \text{ €} \approx 1,500 \text{ €} \quad (5.11)$$

5.5 COSTE TOTAL DEL PROYECTO

Desglose de costes	
Concepto	Importe
Costes de personal	14.950€
Desarrollador Full-Stack	9.750€
Becario Documentación	5.200€
Costes materiales	68,40€
Amortización equipamiento	50€
Despliegue Railway (4 meses)	18,40€
Costes indirectos	1.500€
TOTAL PROYECTO	16.518,40€

Cuadro 5.2: Coste total del proyecto Scubex

PARTE III

DESARROLLO DEL PROYECTO

ARRANQUE

The best way to get started is to quit talking and begin doing.

Walt Disney (1901–1966),

Empresario y cineasta

***E**ste capítulo define la lista inicial de características priorizadas del proyecto Scubex y, a partir de ellas, la arquitectura base necesaria para soportarlas. Se sigue el orden natural del ciclo FDD: primero se establece qué se va a construir, y solo entonces se diseña cómo.*

6.1 LISTA DE CARACTERÍSTICAS

Siguiendo la fase 1 de FDD (Desarrollar lista de características) y los objetivos definidos para el proyecto, se han identificado las siguientes funcionalidades nucleares priorizadas por valor para el usuario:

1. **Gestión de Usuarios (OAuth2):** Autenticación delegada mediante Google y persistencia de perfil (nombre, foto). El usuario puede personalizar su nombre y foto de perfil desde la página de perfil, con soporte para eliminar la cuenta.
2. **Mapa Interactivo:** Visualización de un mapa navegable mediante MapLibre GL JS como superficie central de interacción, con búsqueda de lugares integrada (Nominatim).
3. **Escáner de Especies:** Identificación de fauna marina en una zona mediante radio de búsqueda y validación cruzada de APIs científicas (OBIS + iNaturalist), con sistema de caché adaptativo (TTL 48h) para reducir latencia y llamadas externas.
4. **Datos Climáticos y Oceanográficos:** Condiciones de la zona bajo demanda mediante Open-Meteo (viento, temperatura del agua, corrientes, olas, datos atmosféricos), con evaluación de aptitud para el buceo y caché de 30 minutos en el backend.
5. **Registro de Publicaciones:** Publicación de inmersiones en el mapa con título, descripción, fotografía y geolocalización inversa (nombre del lugar mediante Nominatim). Edición y borrado por el propio autor. Cada publicación genera un enlace compartible (`/map?pub={id}`).
6. **Interacciones Sociales:** Sistema de likes, guardados y comentarios sobre publicaciones. Soporte de menciones en comentarios mediante la sintaxis `@[Nombre] (email)` con notificaciones automáticas al mencionado. El enlace compartible de cada publicación es accesible mediante un botón que usa la Web Share API en móvil y copia al portapapeles en escritorio.
7. **Red Social y Seguimiento de Usuarios:** Perfiles públicos por usuario con recuento de seguidores y seguidos, posibilidad de seguir/dejar de seguir, y vista de publicaciones de cada perfil. Página de perfil propio con mis publicaciones y publicaciones guardadas. Cada perfil dispone de enlace compartible (`/user/{email}`) accesible desde un botón de compartir en la página de perfil.

8. **Sistema de Notificaciones:** Notificaciones en tiempo real (campana en el header) para eventos de seguimiento, like, comentario y mención. Las notificaciones se marcan como leídas al abrirlas o se eliminan permanentemente con swipe (móvil) o botón (escritorio).

Justificación de priorización: La lista anterior refleja el orden de implementación planificado, de mayor a menor prioridad. A continuación se justifica ese orden:

- La autenticación se implementa primero ya que el resto de funcionalidades (avistamientos, perfil, red social) dependen de la identidad del usuario.
- El mapa es la superficie central de la aplicación: sin él no tiene sentido geolocalizar especies, condiciones climáticas ni avistamientos.
- El escáner de especies, los datos climáticos y el registro de publicaciones constituyen el MVP funcional mínimo. Como se planteó en el capítulo de Contexto, el objetivo es que el usuario pueda tomar decisiones apoyándose tanto en datos objetivos como en experiencias reales: el escáner y el clima aportan la primera dimensión; las publicaciones, la segunda, al permitir consultar las inmersiones documentadas por otros buceadores en la misma zona.
- Las funcionalidades de comunidad (likes, comentarios, seguimiento entre usuarios, notificaciones) se implementan sobre ese MVP: convierten la herramienta en una red social de buceadores, pero no son necesarias para que la propuesta de valor principal sea utilizable.

6.2 DISEÑO ARQUITECTÓNICO

Con las características identificadas, el sistema requiere una arquitectura capaz de orquestar múltiples APIs externas, gestionar identidad de usuario y servir una interfaz cartográfica interactiva. Se ha optado por una **arquitectura cliente-servidor desacoplada** con patrón Gateway de APIs.

6.2.1 Stack Tecnológico

- **Backend:** Spring Boot 4 (Java 21). Actúa como Gateway de APIs científicas, orquestador de peticiones asíncronas y gestor de autenticación.

- **Frontend:** React 18 + Vite. Con Hot Module Replacement (HMR) para desarrollo ágil. Gestión de estado reactiva mediante MobX `makeAutoObservable`, con stores separadas por dominio (especies, clima, publicaciones, usuario, mapa).
- **Cartografía:** MapLibre GL JS. Motor de renderizado vectorial para animaciones fluidas y bajo consumo de datos.
- **Base de Datos:** H2 en memoria para el entorno de desarrollo y tests unitarios. PostgreSQL 16 para preproducción y producción, gestionado como servicio en Railway.
- **Testing:** JUnit 5 + Mockito para pruebas unitarias del backend. H2 en memoria como base de datos de test.
- **Seguridad:** Spring Security con OAuth2 Client para Google Login.
- **Documentación de API:** OpenAPI 3.0 (Swagger UI) con anotaciones automáticas.
- **Despliegue:** Railway (backend + PostgreSQL) y Vercel (frontend).

6.2.2 Estrategia de entornos

Se han definido tres entornos que reflejan el ciclo de vida del software desde el desarrollo local hasta la puesta en producción:

Estrategia de entornos de Scubex			
Entorno	Infraestructura	Base de datos	Rama Git
Desarrollo	Local	H2 en memoria	Cualquiera
Preproducción	Railway + Vercel	PostgreSQL (Railway)	develop
Producción	Railway + Vercel	PostgreSQL (Railway)	main

Cuadro 6.1: Estrategia de entornos del proyecto

Justificación de la separación entre plataformas:

- **Vercel** gestiona el frontend React/Vite: Se despliega automáticamente en cada push. Se cuenta con previews por rama sin configuración adicional, más que establecer las direcciones de backend de producción y preproducción.
- **Railway** gestiona el backend Spring Boot y PostgreSQL en el mismo proyecto. Permite tener varios entornos distintos de una misma aplicación desplegados simultáneamente, lo que la hace idónea para combinarla con Vercel.
- La separación permite desplegar frontend y backend de forma independiente. Gracias a esto pueden escalarse independientemente y no solo eso. Debido a la separación se cuenta con mas memoria de los planes gratuitos de ambas aplicaciones, reduciendo costes.

La configuración CORS se ha establecido manualmente mediante la variable `CORS_ALLOWED_ORIGINS` en Railway, y `VITE_API_BASE_URL` en Vercel, configuradas con las respectivas URL del backend correspondientes a preproducción y producción.

6.2.3 Justificación de MapLibre GL JS + MapTiler

La solución cartográfica se compone de dos piezas complementarias:

- **MapLibre GL JS** actúa como motor de renderizado: biblioteca open source (fork de Mapbox GL JS) que renderiza estilos vectoriales mediante WebGL. Permite animaciones fluidas, rotación e inclinación del mapa, y menor consumo de datos frente a tiles rasterizados. Al ser open source, no impone restricciones de licencia sobre el código.
- **MapTiler** provee los tiles y el estilo visual del mapa a través de su API (`VITE_MAPTILER_KEY`). Se ha creado un estilo personalizado (tema marino) alojado en MapTiler Cloud. El plan gratuito cubre de sobra el volumen de peticiones necesario para el proyecto.

La separación entre motor de renderizado (MapLibre) y proveedor de tiles (MapTiler) permite sustituir el proveedor de tiles sin cambiar el código de renderizado, y viceversa.

6.2.4 Integración de APIs Externas

Estrategia de consumo:

APIs externas integradas en Scubex		
API	Proveedor	Datos proporcionados
OBIS	Ocean Biodiversity Information System	Ocurrencias biológicas: nombre científico, coordenadas, taxonomía (phylum), fecha de avistamiento.
iNaturalist	iNaturalist.org	Metadatos multimedia: fotos de especies, nombres comunes (preferred_common_name).
Open-Meteo	Open-Meteo.com	Datos climáticos y oceanográficos: viento, temperatura del agua/aire, altura de olas, corrientes marinas, precipitaciones. API gratuita sin necesidad de clave y con plan gratuito generoso.
Nominatim	OpenStreetMap Foundation	Geocodificación directa e inversa. Directa (/search): buscador de lugares del mapa. Inversa (/reverse): obtención del nombre humano de unas coordenadas (playa, municipio, estado) al crear publicaciones y al visualizar su detalle.

Cuadro 6.2: Resumen de APIs externas

- **OBIS + iNaturalist:** Consultas de especies marinas mediante el botón de escaneo.
- **Open-Meteo:** Consulta bajo demanda al interactuar con el mapa. El backend cachea la respuesta durante 30 minutos (CachedWeather), de modo que peticiones repetidas a las mismas coordenadas dentro de esa ventana no generan llamadas externas.

6.2.5 Patrón Arquitectónico: Gateway API

El backend actúa como director de orquesta de las API, gestionando la conexión entre las mismas. Pasándole al frontend unos datos ya procesados en formato JSON, que el frontend guarda en una store. Este es el flujo de datos desde el principio:

1. **El usuario clicka una zona.** Y react realiza esta llamada GET `/api/species?lat=X&lng=Y&radius=...`

2. **SpeciesService** (backend) realiza:

- a) Una transformación geométrica, convirtiendo el radio a un polígono: (lat , lng , radius) → WKT POLYGON. Esto se realiza porque en la...
- b) Petición a OBIS, hace falta pasar un polígono de la zona. De la cual se extraeran los datos de las especies. La llamada a OBIS es la siguiente: GET /occurrence?geometry=POLYGON(...).
- c) Para cada especie única devuelta por OBIS se realiza un procesamiento de datos:
 - Primero se hace una petición a iNaturalist: GET /taxa?q=<scientificName>. Esta llamada devuelve datos interesantes de la especie que servirán mas adelante.
 - Segundo: Filtrado, si total_results == 0, se descarta la especie por completo. La mayoría de especies que no se encuentran en iNaturalist son microorganismos sin interés para buceadores recreativos.
 - Tercero: Aquí se extraen los datos interesantes. Una foto y nombre común. Para que todas las personas puedan saber que especie se ha registrado

3. Finalmente se unifica la respuesta y se crea un JSON para el frontend. Se hace mediante el DTO SpeciesResponse.

Ventajas:

- React no necesita conocer la existencia de múltiples APIs.
- La lógica de filtrado (microorganismos, especies sin foto) se centraliza en el backend.
- Se evita exposición de API keys en el navegador.

6.3 SEGUIMIENTO Y CONTROL

Fase del proyecto: Arranque (Fases 1-2 de FDD: Lista de características y Modelo Global).

Estado:

OK Arquitectura definida y documentada.

OK Stack tecnológico seleccionado y justificado.

OK APIs externas identificadas y contratos analizados.

OK Backend desplegado en Railway con PostgreSQL. Datasource configurado mediante variables de entorno con H2 para desarrollo local.

OK Frontend desplegado en Vercel. CORS configurado desde variable de entorno en Spring Security y Railway.

OK Escáner de especies funcional en producción: <https://scubex.vercel.app>.

OK OAuth2 implementado.

Decisiones arquitectónicas resueltas en iteraciones posteriores:

- **[Resuelto en It. 2]** Estrategia de caching para OBIS: implementada como entidad `CachedScan` con redondeo adaptativo de coordenadas y TTL de 48 horas. El caching de condiciones climáticas se resolvió con la entidad `CachedWeather` (TTL 30 min).
- **[Postpuesto]** Migración del esquema con Flyway: el proyecto utiliza `spring.jpa.hibernate.ddl-auto=update` en producción. La migración a Flyway se mantiene como mejora futura.

ITERACIÓN 1: ESCÁNER DE BIODIVERSIDAD

*E*sta iteración aborda el MVP de Scubex: La capacidad de identificar especies marinas en una zona determinada e identificarte con OAuth.

7.1 CARACTERÍSTICAS DESARROLLADAS ESTA ITERACIÓN

1. Escáner de Especies con selección de zona en mapa y radio configurable. Ver Tabla §7.1.
2. Enriquecimiento de datos con fotografías y nombres comunes mediante cruce OBIS/iNaturalist.
3. Identificación de usuario con Google OAuth2 y JWT.

7.2 DISEÑO

El diseño se ha centrado en resolver tres incompatibilidades críticas entre el modelo de interacción del usuario y las restricciones de las APIs científicas:

7.2.1 Incompatibilidad 1: Búsqueda radial vs. geométrica

Ejemplo de petición generada:

```
https://api.obis.org/v3/occurrence?  
geometry=POLYGON((-4.0 36.5, -3.4 36.5, -3.4 37.0,  
-4.0 37.0, -4.0 36.5))  
&taxonid=2,3,4  
&size=50  
&fields=scientificName,decimalLongitude,decimalLatitude,  
eventDate,phylum
```

Parámetros clave:

- `geometry`: Polígono WKT cerrado (primer y último punto idénticos).
- `taxonid=2,3,4`: Filtro por phyla (2=Chordata, 3=Arthropoda, 4=Mollusca) para excluir microorganismos unicelulares (aunque no es exhaustivo).
- `fields`: Proyección de campos para reducir payload (omite metadatos innecesarios como `institutionCode`).

7.2.2 Incompatibilidad 2: Calidad de datos visuales

Ejemplo de especie descartada (microorganismo):

OBIS devuelve:

```
{
  "scientificName": "Pycnococcaceae",
  "phylum": "Cyanobacteria",
  ...
}
```

iNaturalist responde:

```
{
  "total_results": 0,
  "results": []
}
```

⇒ **Acción:** Especie filtrada, no incluida en respuesta al cliente.

Ejemplo de especie válida con foto:

OBIS devuelve:

```
{
  "scientificName": "Chimaera monstrosa",
  "phylum": "Chordata",
  ...
}
```

iNaturalist responde:

```
{
  "total_results": 1203,
  "results": [{
    "preferred_common_name": "Rabbit Fish",
    "default_photo": {
```

```

        "url": "https://inaturalist-open-data.s3.amazonaws.com/..."
    }
}
}

```

⇒ **Acción:** Especie enriquecida con foto y nombre común.

7.2.3 Uso del campo `phylum` como fallback

En caso de que iNaturalist devuelva `total_results > 0` pero `default_photo == null`, se usa el campo `phylum` (extraído de OBIS) para asignar un placeholder visual según taxonomía:

- Chordata → Icono de pez genérico.
- Mollusca → Icono de pulpo/calamar.
- Arthropoda → Icono de crustáceo.
- Otros → Icono genérico de organismo marino.

7.3 IMPLEMENTACIÓN

Se ha implementado `SpeciesService.scanSpecies(lat, lng, radius)` que:

1. Convierte coordenadas + radio a formato WKT POLYGON.
2. Consulta OBIS con el polígono generado.
3. Agrupa resultados por nombre científico (deduplicación).
4. Para cada especie, consulta iNaturalist para obtener foto y nombre común.
5. Descarta especies sin foto o sin documentación en iNaturalist.
6. Retorna lista ordenada por número de ocurrencias.

Endpoint REST: `GET /api/species?lat={lat}&lng={lng}&radius={radius}`

7.4 PRUEBAS

Se han implementado los siguientes casos de prueba antes del código productivo:

- **shouldGenerateValidWKTPolygon:** Verifica que, dado un punto (36.5, -4.0) y radio 5000m, se genera un String POLYGON(. . .) geoméricamente cerrado (primer y último punto idénticos).
- **shouldFilterMicroorganismsWithZeroResults:** Mockea una respuesta de OBIS con *Pycnococcaceae* y simula `total_results=0` en iNaturalist. Verifica que la lista final de `SpeciesResponse` está vacía.
- **shouldEnrichSpeciesWithPhoto:** Mockea OBIS devolviendo *Chimaera monstrosa* e iNaturalist devolviendo foto + nombre común. Verifica que `SpeciesResponse.imageUrl` no es null y `commonName == Rabbit Fish`.
- **shouldHandleINaturalistTimeout:** Simula timeout en petición a iNaturalist. Verifica que el sistema descarta esa especie y continúa procesando las demás (resiliencia).

7.4.1 Caché de escaneos (resolución de deuda técnica)

La latencia identificada durante las pruebas (3–7 segundos por escaneo) se resolvió en esta iteración mediante un sistema de caché persistente en base de datos.

Pseudocódigo del flujo con caché:

Identificador	Descripción de la acción de alto nivel			
003	SpeciesService			
Métodos de alto nivel				
List<SpeciesResponse> getSpeciesInSelectedArea(lat:double, lng:double, radius:double)				
Pasos (Usar Pseudocódigo o similar)				
1. roundedLat $\leftarrow$ roundCoord(lat, radius)				
2. roundedLng $\leftarrow$ roundCoord(lng, radius)				
3. cutoff $\leftarrow$ now() $-$ 48h				
4. cached $\leftarrow$ repository.findFirst(roundedLat, roundedLng, radius $\geq$ requested, createdAt > cutoff)				
5. SI cached existe:				
5.1. SI cached.radius > radius: filtrar especies por distancia haversiana $\leq$ radius				
5.2. Devolver lista (instantáneo, sin llamadas externas)				
6. SI NO: ejecutar pipeline completo OBIS $\rightarrow$ iNaturalist				
7. Persistir nuevo CachedScan con especies enriquecidas				
8. Devolver lista				
Métodos de bajo nivel necesarios				
Paso	Clase	Método	Mem. Técn.	IU
1-2	SpeciesService	double roundCoord(value:double, radius:double)	003	SI
4	CachedScanRepository	<CachedScan> findFirst(...)	003	SI

7.4.2 Documentación de API (Swagger/OpenAPI)

Se ha integrado Swagger/OpenAPI 3.0 con springdoc-openapi-ui:

- **Interfaz interactiva:** Swagger UI en <http://localhost:8080/swagger-ui.html>
- **Especificación:** JSON/YAML generado automáticamente desde anotaciones
- **Validación:** Parámetros validados mediante anotaciones @Parameter
- **Respuestas:** 200 OK con lista de especies, 400 Bad Request para coordenadas inválidas

7.4.3 Integración frontend (TD2)

El frontend React consume el endpoint mediante:

- **Componentes:** `MapView.tsx` (mapa interactivo con MapLibre GL JS), `MapRender.tsx` (renderizado de círculo GeoJSON y gestión de clicks), `SpeciesPanel.tsx` (resultados), `ScanningAnimation.tsx` (animación de radar con barrido rotatorio y puntos sincronizados)
- **Estado:** MobX gestiona especies, carga, errores, centro de escaneo y radio configurado (`MapStore` + `SpeciesStore`)
- **Flujo:** Click en mapa → fija centro de escaneo y dibuja círculo GeoJSON → usuario selecciona radio (500 m–10 km) → pulsa *Escanear* → animación de radar con puntos en sector sincronizado → petición HTTP → actualización estado → renderizado panel
- **Animación:** Barrido rotatorio (3 s/vuelta) implementado con Framer Motion. Los puntos de detección se generan cada 80 ms únicamente en un sector de 45° justo detrás de la línea de barrido, usando `map.project()` para convertir coordenadas geográficas a píxeles. Los puntos desaparecen tras una rotación completa.

7.4.4 Configuración del proyecto (TD3)

Backend (Spring Boot):

- **Puerto:** 8080 para el servidor.
- **APIs externas:** URLs de OBIS, iNaturalist y Open-Meteo configuradas en `application.properties`.
- **Base de datos:** H2 en memoria (`jdbc:h2:mem:scubexdb`) para desarrollo rápido sin persistencia.
- **Seguridad:** CSRF deshabilitado (API REST), endpoints públicos (`/api/species`, `/swagger-ui`). Google OAuth2 implementado con validación de tokens mediante `googleapis.com/tokeninfo` y generación de JWT (HMAC-SHA256, 7 días de expiración) con `JwtAuthFilter`.
- **CORS:** Frontend local (`localhost:5173`) permitido para desarrollo.

Frontend (React + Vite):

- **Puerto:** 5173 con Hot Module Replacement.

- **Proxy:** Peticiones a `/api` redirigidas a `localhost:8080` (evita CORS).
- **Gestión de estado:** MobX para estado reactivo de especies y carga.

7.5 DESPLIEGUE

Tras la finalización de la iteración 1, el proyecto fue desplegado en la infraestructura de producción descrita en el capítulo de Arranque:

- **Backend:** desplegado en Railway. El proceso de arranque ejecuta automáticamente `mvn package` y lanza el JAR resultante. Las variables de entorno (`GOOGLE_CLIENT_ID`, `JWT_SECRET`, `DATABASE_URL`, `CORS_ALLOWED_ORIGIN`) se configuran desde el panel de Railway.
- **Frontend:** desplegado en Vercel. Cada push a la rama `main` desencadena un nuevo despliegue automático. La variable `VITE_API_BASE_URL` apunta a la URL pública del backend en Railway.
- **Base de datos:** PostgreSQL gestionado como servicio en Railway; accedido mediante `DATABASE_URL` inyectada en `application.properties`.

La aplicación es accesible públicamente en `https://scubex.vercel.app` desde la finalización de esta iteración.

7.6 SEGUIMIENTO Y CONTROL

Fase del proyecto: Iteración 1 (Fases 3-5 de FDD).

Estado:

OK Endpoint `/api/species` implementado y testeado.

OK Transformación geométrica (WKT Polygon) funcionando.

OK Filtrado de microorganismos operativo.

OK Enriquecimiento con iNaturalist completo (foto + nombre común).

OK Flujo de escaneo en dos pasos (click + botón) con radio configurable (500m–10 km).

- OK Animación de radar con barrido rotatorio y puntos sincronizados en sector.
- OK Google OAuth2 con JWT implementado (login, persistencia en `localStorage`, página de perfil).
- OK Despliegue en Railway (backend) y Vercel (frontend) operativo desde el cierre de esta iteración.
- OK Caché adaptativa de escaneos (`CachedScan`, redondeo por radio, TTL 48 h) implementada — ver §§7.4.1.

Métricas:

- **Cobertura de tests:** 87% en `SpeciesService`.
- **Tiempo de respuesta (promedio):** 3.2s para radio de 5km.

Decisiones técnicas resueltas:

- **[Resuelto]** Caching de taxonomías: implementado como `CachedScan` con redondeo adaptativo por radio y TTL 48 h (ver §§7.4.1).
- **[Postpuesto]** Paginación del endpoint: la limitación actual de 50 especies es suficiente para el caso de uso de buceadores recreativos. Se mantiene como mejora futura.

Análisis de valor aportado: Escáner de Especies	
Propuesta	Endpoint REST que agrega datos de OBIS e iNaturalist para una zona geográfica definida por radio, con interacción en dos pasos (click para situar centro + botón <i>Escanear</i>) y radio configurable (500 m – 10 km).
Valor	Permite al usuario descubrir fauna marina real validada científicamente con representación visual, sin conocimientos técnicos ni acceso directo a bases de datos especializadas. El flujo de dos pasos evita escaneos accidentales.
Coste	Alto esfuerzo de integración. Requiere orquestar llamadas asíncronas, transformar formatos geométricos (coordenadas → WKT Polygon), gestionar inconsistencias entre APIs y aplicar filtrado de calidad. En frontend: círculo GeoJSON en MapLibre, animación radar con Framer Motion.
Opciones	Opción 1: Usar solo OBIS reduciría el coste, pero eliminaría el valor visual (fotos) y semántico (nombres comunes). Opción 2: Usar solo iNaturalist (solo observaciones ciudadanas, sin validación científica OBIS). Opción 3: Escaneo automático al hacer click (descartada: generaba escaneos accidentales).
Riesgos	Rate limits de APIs externas (OBIS: sin límite documentado, iNaturalist: 100 req/min). Inconsistencia de datos (especies en OBIS sin coincidencia en iNaturalist). Latencia acumulada por múltiples peticiones HTTP.
Deuda técnica	Ausencia de caching (cada escaneo regenera peticiones a APIs). Falta de paginación (actualmente limitado a 50 especies). Gestión de errores distribuidos no exhaustiva (si iNaturalist falla, se pierden datos visuales).

Cuadro 7.1: Análisis de valor aportado: Escáner

Memorando técnico 001: Geometría de Búsqueda	
Asunto	Incompatibilidad de búsqueda radial en API OBIS.
Resumen	Generación de polígonos WKT a partir de coordenadas centrales y radio.
Factores causantes	El frontend solicita datos basados en un punto central (lat, lng) y un nivel de zoom (que determina el radio de visión), pero OBIS requiere un objeto POLYGON en formato WKT (Well-Known Text). La API no acepta parámetros centerLat, centerLng, radius.
Solución	Se ha implementado GeometryUtils.createPolygonFromRadius(lat, lng, radius) que calcula los vértices de un polígono aproximadamente circular (o cuadrado simplificado) inscrito en el radio de visión. El resultado se serializa en formato WKT: POLYGON((x1 y1, x2 y2, ..., x1 y1)).
Motivación	OBIS es la mayor base de datos abierta de biodiversidad marina (>150M registros). Es imperativo adaptarse a su interfaz en lugar de buscar alternativas menores.
Alternativas	Filtrado post-proceso: Pedir un área muy grande (bounding box global) y filtrar en memoria por distancia euclidiana. Descartado por ineficiencia (transferencia de datos innecesaria) y riesgo de exceder límites de respuesta de OBIS (size=10000 máximo).
Cuestiones abiertas	Actualmente se utiliza un polígono cuadrado. Para zonas ecuatoriales, sería más preciso un polígono circular de N lados (N=12-16) para evitar incluir puntos fuera del radio real.

Cuadro 7.2: Memorando técnico 001: Geometría

Memorando técnico 002: Calidad de Datos	
Asunto	Presencia de microorganismos y especies sin representación visual.
Resumen	Filtrado cruzado mediante existencia de recursos fotográficos en iNaturalist.
Factores causantes	OBIS devuelve todo tipo de registros biológicos, incluyendo: (1) Microorganismos unicelulares (ej. <i>Pycnococcaceae</i>), (2) Especies con nombre científico válido pero sin observaciones fotografiadas (ej. <i>Thenaea muricata</i>). Para una app turística/deportiva, estos datos no aportan valor experiencial.
Solución	Para cada nombre científico devuelto por OBIS, ejecutar una petición a iNaturalist: <code>GET /taxa?q=<scientificName>&per_page=1</code> . Aplicar los siguientes filtros: (1) Si <code>total_results == 0</code> , descartar especie (no documentada en iNaturalist). (2) Si <code>default_photo</code> es null, descartar especie (sin representación visual). (3) Si ambos criterios se cumplen, extraer <code>preferred_common_name</code> y <code>default_photo.url</code> para enriquecer el modelo de respuesta.
Motivación	Mantener la relevancia visual y la experiencia de usuario. Un buceador no puede "ver" bacterias submarinas, por lo que incluirlas sería ruido informativo.
Cuestiones abiertas	El aumento de latencia por N peticiones HTTP a iNaturalist (N = número de especies únicas en OBIS). Para áreas con >50 especies, el tiempo de respuesta puede superar los 5 segundos. Posible optimización: batching de peticiones o caching de taxonomías previamente consultadas.

Cuadro 7.3: Memorando técnico 002: Filtrado

Memorando técnico 003: Caché Adaptativa de Escaneos	
Asunto	Latencia de 3–7 segundos por escaneo debido a llamadas en-cadenadas a OBIS e iNaturalist.
Resumen	Sistema de caché persistente en PostgreSQL con redondeo adaptativo de coordenadas y TTL de 48 horas.
Factores causantes	Cada escaneo requiere: (1) una llamada a OBIS para obtener ocurrencias marinas, y (2) una llamada a iNaturalist por cada especie única detectada (N llamadas secuenciales/paralelas). Para zonas costeras con alta biodiversidad, N puede superar las 40 especies, acumulando varios segundos de latencia de red.
Solución	Se ha implementado la entidad CachedScan que almacena el resultado completo de un escaneo indexado por (rounded-Lat, roundedLng, radius). El redondeo de coordenadas es adaptativo según el radio solicitado: $\text{radio} \leq 1000 \text{ m} \rightarrow 0.01^\circ$ ($\approx 1.1 \text{ km}$ de tolerancia); $\text{radio} \leq 2000 \text{ m} \rightarrow 0.05^\circ$ ($\approx 5.5 \text{ km}$); $\text{radio} > 2000 \text{ m} \rightarrow 0.1^\circ$ ($\approx 11 \text{ km}$). Adicionalmente, una caché con radio mayor sirve para satisfacer una consulta de radio menor (filtrando por distancia haversiana), maximizando los hits.
Motivación	Reducir el número de llamadas a las APIs externas y conseguir respuesta instantánea para zonas ya consultadas. Las especies marinas de una zona no cambian en un horizonte de 48 horas, por lo que el TTL elegido garantiza frescura sin impactar la experiencia de usuario.
Cuestiones abiertas	El redondeo puede hacer que dos clicks muy próximos compartan la misma caché, lo que es el comportamiento deseado. En zonas fronterizas (p.ej., entre 0.04 y 0.06 grados) puede haber un falso miss si el usuario hace click justo en el umbral de redondeo.
Alternativas	Spring Cache con backend Redis: descartado por añadir un servicio externo de infraestructura. La solución en PostgreSQL es más simple y ya disponible en Railway.

Cuadro 7.4: Memorando técnico 003: Caché de Escaneos

PARTE IV

CIERRE DEL PROYECTO

MANUAL DE USUARIO

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

8.1 SECCIÓN LIBRE

Estructurar en función del proyecto.

CONCLUSIONES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

9.1 INFORME POST-MORTEM

Qué es un informe post-mortem

9.1.1 Lo que ha ido bien

- Argumento a favor 1.
- Argumento a favor 2.
- Argumento a favor 3.

9.1.2 Lo que ha ido mal

- Argumento en contra 1.
- Argumento en contra 2.
- Argumento en contra 3.

9.1.3 Discusión

En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.

9.2 TRABAJOS FUTUROS

Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.

PARTE V

APPENDICES

SOFTWARE PRODUCT LINES

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

*Ernest Hemingway (1899–1961),
Novelist*

***T**his is an example of an abstract. Multiple lines are supported. Several paragraphs. It jumps to the next page. Blau blau blau. I am introducing more text to reach the third line*

A.1 SOFTWARE PRODUCT LINES

- Objective of a Product Line (PL) (mass production and customisation) [1]
- The focus in software derives in Software Product Lines (SPLs).
- Variability management: variability models
- When and how are used VMs: FMs are described in FODA report as a key element in SPL since they represent the variability and commonality of the different products in a SPL.

A.2 FEATURE MODELS

To Abductive Section in 2.1

As the number of products to be built by a SPL may be large and the constraints among features may be complex, representing such an information in a manageable and compact manner is a must. Feature Models (FMs) represent the set of products a SPL may build in terms of product features. Some features are optional while others are mandatory. To indicate the relationships among features, they are hierarchically linked, forming a tree whose root is a feature representing the whole functionality of a product. The root feature is refined in child features, which increase the level of detail and reduce the scope of features. Recursively following this refinement process, a tree-like structure is obtained where three basic kinds of hierarchical relationships are used:

- **Mandatory:** a mandatory relationship affects a parent and child feature. It forces the child feature to appear in a product whenever its parent feature does.
- **Optional:** a child feature connected to a parent feature by means of an optional relationship may be optionally selected whenever its parent feature is.
- **Set-relationships:** three or more features are part of a set-relationship: a parent feature and a set of two or more child features. A set-relationship contains a cardinality that constraints the number of child features to be selected in a product whenever its parent feature is selected. If the cardinality is $[1,1]$ it is commonly remarked as an *alternative relationship* where only one child feature may be selected at the same time. If the cardinality is $[1..N]$ (where N is the number of

child features), it is also known as an *or-relationship* as any combination of child features is allowed while at least one is selected.

Although FMs can represent most of the most frequent constraints, the hierarchical nature of these models might hinder the representation of some constraints. Under this circumstance, *cross-tree constraints* can be added. The most common kinds of cross-tree constraints are:

- Dependency: a feature depends on another feature if the second one must be part of a product whenever first one is selected.
- Exclusion: two features exclude themselves if both of them cannot be part of a product at the same time.

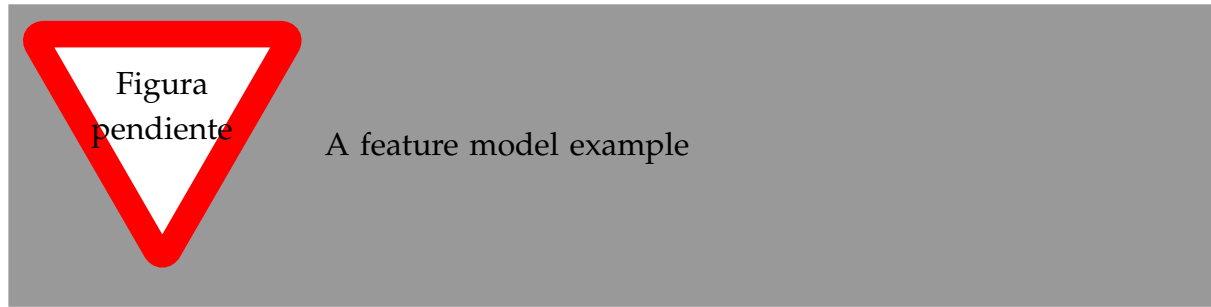


Figura A.1: An example of a Home Integration System

The example in Figure §A.1 describes a *Home Integration System* (HIS) SPL in terms of its features and the relationships among them. Leaning on this example we define some useful terms:

Partial configuration : a partial configuration is a composed by three sets of selected (S), removed(R) and undecided(U) features. A feature can only be in one of these sets and every feature in the FM (fm) must be in one of them, i.e. $S \cup R \cup U = fm$ and $S \cap R \cap U = \emptyset$. A partial configuration represents an intermediate state during the process of a customer selecting the feature for a custom product. For example, $S_P = \{\dots\}$, $R_P = \{\dots\}$ and $U_P = \{\dots\}$ define a partial configuration for the sample FM where some features are still to be decided if they are to be selected or removed in a configuration.

(Full) configuration : a full configuration or simply a configuration is a partial configuration such that the set of undecided features is empty. For example, $S_F = \{\dots\}$ and $R_F = \{\dots\}$ describe a full configuration for the example FM.

Product : a product is a representation for a full configuration such that only the selected features are remarked. For instance, $P = \{\}$ is a product for the above full configuration. A product such as A,B is a valid since all the constraints within the FM are satisfied. However, A,B and C is not a valid product since D is required.

Validation A partial configuration is *valid* if all the relationships and constraints are satisfied given the sets of selected, removed and undecided features. So the definition applies for valid full configurations and valid products. As a conclusion we can affirm that a FM represents all the valid products in a SPL.

Objetivo: Briefly expose attributes as an important asset in feature models.

It is frequent that features are not enough to represent information that is relevant to represent a SPL variability. In this case, FMs are extended with feature attributes such as cost, versions, RAM consumption, etc. in the so-called Extended Feature Models (EFMs) [1]. Besides relationships, an EFM contains constraints that affect attributes which reduce even more the set of products a FM describes. Above definitions remain when attributes are introduced into FMs.

A.3 AUTOMATED ANALYSIS OF FEATURE MODELS

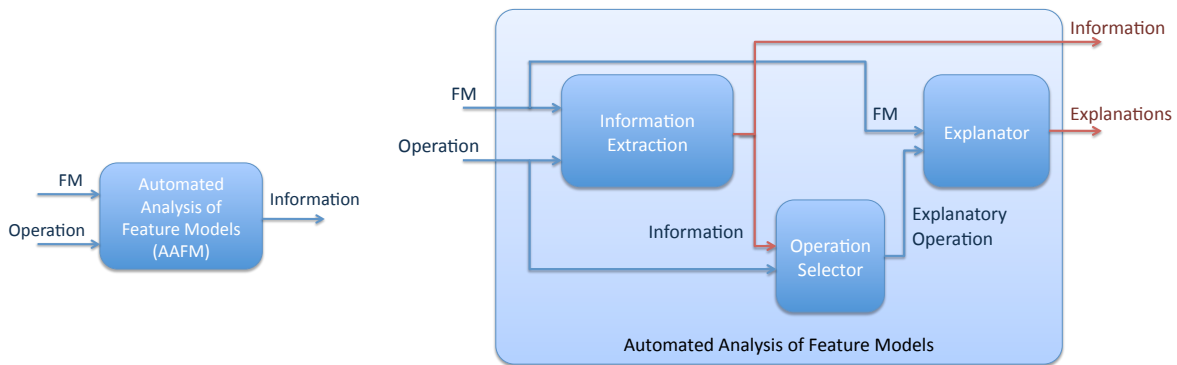
A.3.1 Scope

To Abductive Intro

FMs are used all along the SPL development as key models and many of the development decisions are taken relying on the information contained within them. Most of the times, relationships are complex and hinder the manual extraction of information. Manually obtaining information such as ‘which is the product that costs the less?’, ‘does the feature model contain errors?’ or ‘why there exist no product containing certain features?’ can be an unfeasible task. The complexity and compactness of FMs justify the need of an automated support of these operations. So the *Automated Analysis of Feature Models* (AAFM) arises as a topic of interest to deal with this problem in the SPL community.

The AAFM can be seen as a black-box process that receives a FM and an operation as inputs and obtains information (its kind depends on the analysis operation) as an

output (Fig. A.2(a)). There are many operations that extract information from a FM such as 'counting products' operation whose result is a natural number indicating the number of customised products that can be built; or 'list of products' operation that obtains each of those products. This vision of AAFM as a black-box is valid for a subset of analysis operations that we call *information extraction operations* (IEO) that can be seen as processes to extract information from FMs. In other words, an IEO makes explicit an implicit information within a FM.



(a) The AAFM seen as a black-box process

(b) Extending the AAFM process with explanations

Figura A.2: A different view on AAFM distinguishing between information extraction and explanatory operations

Use me to explain in a larger text than 'side-text' anything that is important to a reader not familiar with the dissertation context for example.

However, there is a subset of analysis operations known as *explanatory operations* (EO) whose objective is explaining the result obtained from a IEO. Sometimes, the result is not the expected one and the analyser needs to know which are the relationships that have caused it. For example, let us suppose that the IEO 'which are the products described in a FM that cost less than \$1000?' obtains no products as a result. If we were expecting to obtain at least one product, it is important to determine the relationships in the FM that are responsible of that behaviour, so an EO 'why there is no product costing less than \$1000?' will shed light on the relationships that avoid obtaining any product. Obtaining no result is not the only case that claims for explanations.

If we obtained only one product as a result and we were expecting to obtain at least 10 products, although an answer is obtained the result is unexpected and the discrepancy reasons have to be found. Moreover, explanatory operations are also use-

ful even when an expected result is obtained, to reinforce the certainty that the result is correct. So it can be concluded that EOs complement the information an FM analyser obtains from IEOs.

The complexity of feature modelling relies on correctly setting the relationships that describe the set of products to be built in a SPL. Relationships are the only elements responsible of the results obtained in FM analysis. So an *explanation* is a set of relationships that may have caused that result. While IEO provides for an unique response that is known for certain, an EO provides for a set of probable explanations to a result obtained from a IEO, being only one of them a valid explanation. It would be the analyser the one in charge of discriminating the correct explanation, maybe performing new analysis operations.

**THIS IS A SIDE TEXT. USE TO
REMARK IMPORTANT
INFORMATION**

Therefore, two kinds of operations are distinguished in AAFM: information extraction and explanatory operations. Explanatory operations have no sense without a paired information extraction operation and its result. To ensure that explanatory operations are always paired to an information extraction operation, we define a new black-box process of AAFM that incorporates explanations as an additional output (see Figure A.2(b))

1. Information extraction: the original process, which remains the same.
2. Operation selector: depending on the information extraction operation the analyser asks for and the information obtained as a result, this process provides the explanatory operation to be performed. In other words, it pairs an explanatory operation to an information extraction operation.
3. Explanatory analysis: provides a set of explanations from the FM and the explanatory operation.

The overall process can be encapsulated into a holistic black-box process which receives the FM and the information extraction operation as inputs and provides a result and explanations as outputs. It can be seen as we just add explanations as an output to the analysis process.

To realise this view on the AAFM, we need to give details on the insides of these black-boxes. Since the information extraction process is already rigourously defined in

Benavides' PhD dissertation, the purpose of this paper is defining the remaining two sub-processes. We formalise the explanatory analysis process by means of default logic and provide the criteria to implement the operation selector process.

Most Common Techniques to perform AAFM Operations.

A.4 DYNAMIC SOFTWARE PRODUCT LINES (DSPL)

What is a Dynamic Software Product Line (DSPL). Different points of view. What is important is the automation of reconfiguration properties relying on SPL techniques.

We focus in the application of explanations in DSPLs as an application of our results. Specifically we have worked in MAS and smart homes providing a solution for automating product reconfiguration.

A.5 HYPOTHESIS AND OBJECTIVES

Objetivo: Justifying that explanations are a particular set of operations in AAFM that are not solvable by means of the techniques that are used up-to-date

Objetivo: Set an impacting phrase that summarises the hypothesis

Hypothesis

*Explanations cannot be solved by AI techniques used to solve AAFM.
There should exist other AI techniques to solve explanations.*

Objective of the dissertation

Defining a framework to provide solutions for explanatory analysis in FMs.

This dissertation summarises our contribution to solve some of the objectives we set in our PhD project.

- Defining a catalog of analysis operations where explanations are applied.
- Rigorously defining these operations in terms of logics.
- Proposing solutions to these operations.
- Validating our results by means of tools and projects where they are applied.

Next chapter focuses on refining how we have contributed to deal with the above objectives.

A piece of code...

```
public Map<Cardinality, CardinalValue> detectWrongCardinals() {
    // any other implementation of Map can be used instead.
    Map<Cardinality, CardinalValue> result =
        new TreeMap<Cardinality, CardinalValue>();
    for( r : relationships) {
        if (r instanceof Set) {
            Set set = (Set)r;
            Cardinality card = set.getCardinality();
            Domain dom = card.getDomain();
            for (value: dom.getValues())
                if (isWrongCardinal(card, value))
                    result.put(card, value);
        }
    }
    return result;
}
```

A coolTable. Use inside a table.

Use `\TableSubtitle{n,title}` to add a subtitle as the header. *n* is the number of columns and *title* is the text to place. [1]

A Catalog of FM Explanatory Operations (2009 version)		
Information Extraction Operation	FM Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid FM	-	invalid FM
Valid Configuration	valid partial conf.	invalid partial conf.
Valid Product	valid product	invalid product
Products Listing	vaild Product/Config	invalid FM/Product/Config
Products Counting	vaild Product/Config	invalid FM/Product/Config
Optimisation	vaild Product/Config	invalid FM/Product/Config
Core feature	core feature	core feature
Variant feature	variant feature	variant feature
Dead feature detection	-	dead feature
False-optional feature detection	-	false-optional feature
Wrong-cardinality detection	-	wrong cardinal
Information Extraction Operation	Configuration Explanatory Operations	
	<i>Why? operation</i>	<i>Why not? operation</i>
Valid Configuration	valid partial conf.	invalid partial conf.

Cuadro A.1: Most frequently used explanatory operations and their corresponding information extraction operations

BIBLIOGRAFÍA

- [1] D. Benavides, A. Ruiz-Cortés, and P. Trinidad. Automated reasoning on feature models. *LNCS, Advanced Information Systems Engineering: 17th International Conference, CAiSE 2005*, 3520:491–503, 2005. ISSN 0302-9743. (pages 60, 62 y 66).